

오늘의 작업기록 — 2026년 7월 6일

ledger-api — 첫 HTTP 서버(Spring Boot) 부트스트랩 (m105 조각 1)

(순수 Java를 졸업하고, 죽지 않고 요청을 기다리는 프로세스를 처음 다룬 날)

한눈에

항목	내용
목표	m105 조각 1 — 첫 Spring Boot API: 생성 → 컨트롤러 → 부팅 → 검증 → push
결과	완료 ✓ — curl 200 OK(16바이트) 검증 + 잔디 계정 인정(login) 확인
repo	신규 Taewoo-Won/ledger-api (public) — mini-ledger와 별개 프로젝트
커밋	94c794a "bootstrap ledger-api with health endpoint (verified: 200 OK)" — 13 files, 475 insertions
환경 명세	Java 21.0.11 · Gradle 9.5.1 · Spring Boot 4.1.0 · Tomcat 11.0.22 · curl 8.5.0 (VPS Ubuntu 24.04)
오늘의 원칙	★ "동작 확인(curl 응답) → 그담 커밋" — 검증 안 된 반쪽을 잔디 때문에 올리지 않는다
새 개념	서버(죽지 않는 프로세스) · package · 애노테이션(@RestController / @GetMapping) · Gradle/bootRun · 백그라운드 & / kill %1 · curl -v
착시 3건	① 16분 EXECUTING=멈춤? → Gradle 최초 다운로드+서버 정상 대기 ② curl 무반응 → 개행 없는 200 OK ③ push=잔디? → API login 으로 확인
다음	조각 2 — mini-ledger의 Bank 를 태워 POST /transfer → 잔액 변경 (m105 완료 조건)

어제 mini-ledger 학습판 Layer 1을 달았다. 오늘은 그 순수 Java 원장을 HTTP로 감싸는 첫 층을 올렸다. mini-ledger는 java Main 한 번 돌고 끝나는 프로그램이었지만, ledger-api는 켜져서 요청을 계속 기다리는 서버다. 실제 핀테크 백엔드가 전부 이 구조 위에 있다 — 앱이 "이체해줘"라고 HTTP로 요청하면 서버가 처리해 응답하는 것. 오늘은 그 뼈대의 첫 조각(/health)이고, 스프링의 마법을 그냥 삼키지 않고 "뭘 대신해주는지"를 목록으로 만들며 갔다.

0. 오늘의 위치와 철학

mini-ledger 졸업 → ledger-api 시작.

- 어제까지 (mini-ledger, m103): 순수 Java. 감사로그·먹등성·조회·환불·정산·복식부기. javac *.java && java Main 으로 한 번 실행. Layer 1 학습판 완성.
- 오늘부터 (ledger-api, m105): Spring Boot. HTTP 서버. 새 폴더(~/ledger-api)·새 repo. mini-ledger는 건드리지 않는다.
- 왜 지금 Spring인가: 한국 핀테크 공고 1순위 스택 = 7월 최대 바위. mini-ledger가 신선할 때 HTTP로 감싸는 게 최적의 다음 층.

핵심 철학 — 마법을 목록화한다. Spring은 파일 몇 개로 웹서버가 뜨는 마법처럼 느껴진다. 마법을 그냥 삼키지 말고 "스프링이 대신해주는 것"을 명시한다. 이 마법의 아래층 (요청 받는 서버, 트랜잭션)은 8월에 직접 만든다 — 그게 이 포트폴리오의 정체성이다.

오늘의 마법 목록 3개 (+ 각각이 눈에 보인 순간):

#	스프링이 대신해준 것	오늘 로그에서 보인 증거
①	내장 웹서버(Tomcat) — 8080 포트를 열고 대기	Tomcat started on port 8080 (http)
②	컴포넌트 스캔 — 클래스를 훑으며 @ 스티커로 역할 부여	@RestController 만 붙었는데 URL이 살아남
③	URL→메서드 자동 연결 — GET /health → health() 호출	curl 한 방에 200 OK + 16바이트 응답

재현 타임라인 (오늘 실제 순서 · UTC)

```
10:58:38 ./gradlew bootRun (포그라운드, 최초) — Gradle 9.5.1 다운로드 ~16분 포함
10:58:41 Tomcat started on port 8080 · Started in 4.321s (java PID 1152961)
~11:15 새 탭 못 열어 Ctrl+C로 종료 → 백그라운드 재기동 결정
11:16:08 bootRun 백그라운드(&) 재기동 — Started in 3.964s (java PID 1153317)
11:25:52 curl -v → HTTP/1.1 200 · Content-Length: 16 ← 검증 성공 시각
~11:27 kill %1 → 서버 종료, 포트 8080 닫힘 확인
11:29:08 git commit 94c794a (noreply 이메일) → gh repo create + push
직후 GitHub API로 author.login = "Taewoo-Won" 확인 → 잔디 인정
```

PART 1 — 생성: 프로젝트 뼈대 받기 (start.spring.io)

1-1. 친 명령어

```
mkdir ~/ledger-api && cd ~/ledger-api
curl https://start.spring.io/starter.tgz \
  -d type=gradle-project -d language=java -d javaVersion=21 \
  -d dependencies=web \
  -d groupId=com.ledger -d artifactId=ledger-api \
  -d name=ledger-api -d packageName=com.ledger.api \
  | tar -xzv
```

1-2. 옵션 해석 (이 요청이 뭘 시킨 건가)

- **start.spring.io** = 스프링 프로젝트 **빠대 생성 공식 서비스**. 브라우저로 클릭해서 받는 걸 curl 한 방식으로 요청한 것.
- **-d type=gradle-project -d javaVersion=21** = 빌드 도구는 Gradle, 자바는 21로.
- **-d dependencies=web** = **웹 서버 기능**(HTTP 수신)을 재료로 포함 — 이게 오늘의 본체.
- **-d packageName=com.ledger.api** = 코드 주소 지정 → **src/main/java/com/ledger/api/** 에 패키지 구조 생성.
- **| tar -xzv** = 응답으로 온 압축(.tgz)을 받는 즉시 현재 폴더에 해제.

1-3. 판정 — 생긴 것과 각 역할

```
ls -la ~/ledger-api && ls ~/ledger-api/src/main/java/com/ledger/api/
```

```
build.gradle          ← 의존성/빌드 설정. dependencies=web이 여기 반영 ("재료 목록")
gradlew, gradlew.bat  ← Gradle 래퍼(실행 도구). 이걸로 bootRun 한다
settings.gradle
src/main/java/com/ledger/api/LedgerApiApplication.java ← 자동 생성된 메인 진입점(서버 시작점)
src/main/resources/application.properties
src/test/java/com/ledger/api/LedgerApiApplicationTests.java
```

노이즈로 버림: **tar: Ignoring unknown extended header keyword ...** 다수 — macOS 계열 메타데이터를 무시했다는 무해 경고. 파일은 전부 정상 해제됨. 재현에 무관.

PART 2 — 컨트롤러: /health 엔드포인트 작성

2-1. 친 명령어 + 작성한 플코드

```
nano src/main/java/com/ledger/api/HealthController.java
```

작성한 전체 코드:

```
package com.ledger.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {

    @GetMapping("/health")
    public String health() {
        return "ledger-api alive";
    }
}
```

2-2. 새 문법 3개 — 뭐가 처음이고, 스프링이 이걸로 뭘 하나

- ① **package com.ledger.api;** — 파일의 "주소" - 이 파일이 **com/ledger/api/** 폴더에 있다는 선언. **실제 폴더 경로와 반드시 일치**해야 한다. - mini-ledger는 패키지 없이(default) 썼다. 실무 코드는 전부 패키지로 정리 — 오늘이 첫 사용.
- ② **@RestController** — 스프링이 읽는 "스티커"(애노테이션) - **@** 로 시작하는 표식. 사람이 아니라 **스프링이 읽는다**. - 서버가 뜰 때 스프링이 클래스들을 훑으며(컴포넌트 스캔 = 마법②) 이 스티커를 발견 → "이 클래스는 HTTP 요청 담당"으로 등록한다. 내가 등록 코드를 한 줄도 안 썼는데 등록되는 이유가 이것.
- ③ **@GetMapping("/health")** — **URL을 메서드에 못박기** - "GET /health 요청이 오면 이 메서드를 실행해"(= 마법③). - **return "ledger-api alive"** = 그 문자열이 곧 **HTTP 응답 본문**. println이 아니라 return이 응답이 된다 — 이게 mini-ledger와 사고방식이 다른 지점.

→ 최소 골격: **@RestController**(이 클래스가 담당) + **@GetMapping**(이 URL은 이 메서드) + **return**(응답 내용).

2-3. 판정 — 파일 확인 (grep 산수 포함)

```
cat src/main/java/com/ledger/api/HealthController.java # 전체 내용 정상
grep -c "GetMapping\|RestController\|package com.ledger.api" \
    src/main/java/com/ledger/api/HealthController.java
# → 5
```

5가 정답인 이유: import 2줄(GetMapping·RestController) + 애노테이션 2줄(@RestController·@GetMapping) + package 1줄 = 5줄. 세 "종류"가 아니라 그 단어가 등장한 "줄 수"를 세기 때문. (처음에 3을 기대했다가 5가 나와 확인한 것 — 산수가 맞았다.)

PART 3 — 부팅: bootRun으로 서버 기동

3-1. 천 명령어와 실제 출력 (버전 명세 포함)

```
cd ~/ledger-api && ./gradlew bootRun
```

```
Fetching distribution.
Downloading https://services.gradle.org/distributions/gradle-9.5.1-bin.zip
.....10% ... 100%
Welcome to Gradle 9.5.1!

...
:: Spring Boot ::                (v4.1.0)

... Starting LedgerApiApplication using Java 21.0.11 ... (PID 1152961)
... Tomcat initialized with port 8080 (http)
... Starting Servlet engine: [Apache Tomcat/11.0.22]
... Tomcat started on port 8080 (http) with context path '/'
... Started LedgerApiApplication in 4.321 seconds
```

성공 판정 2줄: Tomcat started on port 8080 + Started LedgerApiApplication. 이 둘이 뜨면 서버가 산 것이다.

(만약 Permission denied 가 났다면 chmod +x gradlew 후 재시도 — 오늘은 안 걸림.)

3-2. 트러블슈팅 A — "16분째 EXECUTING, 멈춘 거 아니냐"

증상: 성공 로그 뒤에 화면이 이렇게 굳은 듯 보임:

```
|■■■■■■■■■■···| 80% EXECUTING [16m 18s]
> :bootRun
> IDLE
```

원인 (두 겹): 1. 16분의 대부분 = Gradle 배포판 최초 다운로드. gradle-9.5.1-bin.zip 을 받는 시간이지 서버가 느린 게 아니다. 딱 한 번만 걸린다 — 실제로 두 번째 기동은 3.964초. 2. EXECUTING [16m] 은 "16분째 정상 실행 중"이라는 뜻. bootRun 은 서버를 켜놓고 Ctrl+C 전까지 계속 유지한다(요청을 받아야 하니까). IDLE = 처리할 요청 없이 대기 중 = 정상.

교훈: 서버는 원래 죽지 않고 기다린다. java Main (한 번 돌고 종료)만 다뤄본 눈에는 이게 "멈춤"으로 보인다 — 오늘 처음으로 "죽지 않는 프로세스"를 다룬 것이고, 이 낯설 자체가 서버의 본질이다.

PART 4 — 검증: curl로 요청 찢어보기 (오늘의 크릭스)

4-1. 트러블슈팅 B — 서버가 화면을 점유해서 검증할 창이 없다

증상: bootRun 이 터미널을 잡고 있어 그 창에서 curl을 못 쳤. 검증은 다른 창이 필요한데 Termius 새 탭 여는 법을 모름.

해결 — 백그라운드(&)로 재기동해 한 창에서 해결:

```
# 1) 포그라운드 서버 종료
Ctrl + C

# 2) 백그라운드 재기동 (출력은 server.log로, &로 뒤에서 실행)
cd ~/ledger-api && ./gradlew bootRun > server.log 2>&1 &
# → [1] 1153271 (백그라운드 작업번호 [1] + gradle 프로세스 PID)
```

- & = 명령을 뒤에서 돌리고 프롬프트를 즉시 돌려받는다 → 같은 창에서 다음 명령 가능.
- > server.log 2>&1 = 서버 출력(표준+에러)을 화면 대신 파일로.
- 이번 기동은 Gradle이 이미 있어 3.964초에 완료.

서버 준비 판정 (화면 대신 로그를 grep):

```
grep "Tomcat started\|Started LedgerApi" ~/ledger-api/server.log
# → Tomcat started on port 8080 (http) ...
# → Started LedgerApiApplication in 3.964 seconds ... (java PID 1153317)
```

참고: [1] 1153271 (gradle)과 로그의 PID 1153317 (java)이 다른 이유 — gradlew가 자바 프로세스를 자식으로 띄우기 때문. kill %1 은 작업 전체를 내리므로 자식(서버)도 함께 종료된다(4-4에서 확인됨).

4-2. 트러블슈팅 C — curl이 무반응처럼 보였다 (진짜 원인: 개행 없는 200)

증상: 서버 기동을 확인했는데도, curl이 아무것도 안 찍고 프롬프트로 돌아옴:

```
curl http://localhost:8080/health
# → (아무것도 안 보임)
```

가설 1 — "서버가 덜 났다" → 4-1의 grep으로 기동 확인 후 다시 쳐도 여전히 무반응. 가설 기각.

실체 확인 — curl -v (자세히)로 요청/응답을 해부:

```
curl -v http://localhost:8080/health
```

```
> GET /health HTTP/1.1           |
> Host: localhost:8080           | 내가 보낸 요청 (>)
> Accept: */*                     |
< HTTP/1.1 200                   | ← 성공 응답!
< Content-Type: text/plain; charset=UTF-8
< Content-Length: 16             | 서버의 응답 (<)
< Date: Mon, 06 Jul 2026 11:25:52 GMT
```

- > 줄들 = 요청(GET /health를 보냄), < 줄들 = 응답. HTTP 한 사이클이 이 몇 줄이다.
- 200 = 성공. Content-Length: 16 = 본문이 16바이트 — ledger-api alive 가 정확히 16자. 응답은 매번 정상으로 오고 있었다.

진짜 원인: return "ledger-api alive" 에는 개행(\n)이 없다. curl이 16자를 출력했지만 줄바꿈이 없어 다음 프롬프트가 같은 줄에 붙어 안 보였던 것. (wget -q0- 도 똑같이 "안 보였다" — 같은 원인의 방증.)

교훈: "안 보인다 ≠ 안 왔다." 응답의 실체는 -v 의 상태코드와 Content-Length로 판정한다.

4-3. 포트 점유 확인 (추가 판정)

```
ss -tlnp | grep 8080
# → LISTEN 0 100 *:8080 *: * users:(("java",pid=1153317,fd=40))
```

→ 8080을 java(우리 서버)가 LISTEN 중. 서버 정상인 독립 증거.

4-4. 서버 내리기 — 공개 IP 방치 금지

```
kill %1
# → [1]+  Terminated    ... ./gradlew bootRun > server.log 2>&1
ss -tlnp | grep 8080
# → (출력 없음 = 포트 닫힘)
```

왜 바로 내리나: 이 서버는 공개 IP(104.236.232.210) 위에 떠 있고 지금은 보안 설정이 전혀 없다. 켜둔 채 방치하면 인터넷의 누구나 접근 가능. → "검증용으로 잠깐 켜고, 확인되면 끈다."

PART 5 — 잔다: 검증된 것만 올린다 + 어제 이메일 수정 최종 검증

5-1. 순서 원칙

curl 200 OK를 확인한 다음에야 커밋한다. 잔디가 급하다고 검증 안 된 반쪽을 먼저 올리지 않는다 — 증분 1부터 지켜온 순서이고, 오늘도 이 순서대로 갔다.

5-2. init → 이메일 설정 → 커밋 → 이메일 확인

```
cd ~/ledger-api && git init && \
  git config user.email "288912277+Taewoo-Won@users.noreply.github.com" && \
  git config user.name "Taewoo-Won"

git add . && git commit -m "bootstrap ledger-api with health endpoint (verified: 200 OK)"
# → [main (root-commit) 94c794a] ... 13 files changed, 475 insertions(+)
```

★ 커밋 직후 이메일 판정 (어제 교훈 — 가정 말고 확인):

```
git log -1 --format='%ae'
# → 288912277+Taewoo-Won@users.noreply.github.com ✓
```

5-3. GitHub repo 생성 + push (gh)

```
gh repo create ledger-api --public --source=. --push
```

```
✓ Created repository Taewoo-Won/ledger-api on GitHub
✓ Pushed commits to git@github.com:Taewoo-Won/ledger-api.git (29 objects, 50.87 KiB)
HEAD -> main
```

5-4. ★ 잔디 인정 최종 검증 — API에서 login 이 붙었는가

어제 잔디가 안 깎인 원인 = 커밋 이메일이 계정과 불일치(GitHub이 "누구 커밋인지 모름" 처리). **push 성공만 보고 "됐겠지" 하지 않고**, GitHub이 이 커밋을 내 계정 소유로 인식했는지 API로 직접 확인:

```
curl -s "https://api.github.com/repos/Taewoo-Won/ledger-api/commits/main" | grep -A3 '"author"'
```

```
"author": {
  "name": "Taewoo-Won",
  "email": "288912277+Taewoo-Won@users.noreply.github.com", ← 커밋에 박힌 이메일
  ...
"author": {
  "login": "Taewoo-Won", ← ★ GitHub이 계정 소유로 인정했다는 표식
  "id": 288912277, ← 내 계정 ID와 일치
```

→ **"login": "Taewoo-Won" 이 붙었다 = 잔디가 깎인다**. 어제는 이 login이 붙지 않아 안 깎였던 것. 이메일 수정이 실전에서 먹히는 것을 오늘 새 repo의 첫 커밋으로 확정했다.

PART 6 — 오늘 잡은 개념 (복기용)

- 서버 vs 일회성 실행: **java Main** 은 한 번 돌고 종료. 서버는 **켜진 채 요청을 계속 대기**(Ctrl+C 전까지). **EXECUTING [Nm]** · **IDLE** 은 멈춤이 아니라 정상 대기.
- **Spring Boot**: 웹서버를 미리 만들어 제공하는 프레임워크. 오늘 확인한 대항 3가지 — Tomcat 기동 / 컴포넌트 스캔 / URL→메서드 매핑.
- **Gradle / gradlew**: Java 빌드·실행 도구(javac/java 대체). 배포판은 **최초 1회만** 다운로드(오늘 16분의 정체) — 이후 기동 3.9초.
- **Tomcat**: Spring Boot 내장 웹서버. 8080 포트에서 HTTP 수신.
- **package** : 파일의 주소. 폴더 경로와 일치 필수. (mini-ledger는 default였고 오늘 첫 사용.)
- 애노테이션 **@** : 스프링이 읽는 스티커. **@RestController** =HTTP 담당 클래스, **@GetMapping("/x")** =이 URL은 이 메서드, **return 값 = 응답 본문**.
- 백그라운드 실행: **명령 &** = 뒤에서 실행+프롬프트 즉시 반환. **> f 2>&1** = 출력을 파일로. **kill %1** = 작업 1번 종료(자식 프로세스 포함).
- **curl -v** : **>** =요청, **<** =응답을 그대로 보여줌. 본문이 안 보여도 **상태코드(200)**와 **Content-Length**로 응답 실체를 판정.
- 잔디 인정 조건: push 여부와 별개로, 커밋 이메일이 **계정 이메일(또는 noreply)**과 일치해야 GitHub이 소유로 인정(**login** 이 붙음)하고 잔디를 깎는다. 판정은 commits API의 **author.login** .

7. 재현 절차 요약 (6개월 뒤 따라치기용 — 명령만)

```
# 1) 생성
mkdir ~/ledger-api && cd ~/ledger-api
curl https://start.spring.io/starter.tgz -d type=gradle-project -d language=java \
  -d javaVersion=21 -d dependencies=web -d groupId=com.ledger -d artifactId=ledger-api \
  -d name=ledger-api -d packageName=com.ledger.api | tar -xzv

# 2) 컨트롤러 작성 (PART 2-1 풀코드)
nano src/main/java/com/ledger/api/HealthController.java

# 3) 백그라운드 기동 + 준비 판정
./gradlew bootRun > server.log 2>&1 &
grep "Tomcat started\|Started LedgerApi" server.log      # 두 줄 뜨면 준비 완료

# 4) 검증 (200 + Content-Length: 16이면 성공)
curl -v http://localhost:8080/health

# 5) 서버 내림 (공개 IP)
kill %1 && ss -tlnp | grep 8080                        # 출력 없어야 정상

# 6) 검증된 것만 push + 잔디 판정
git init && git config user.email "288912277+Taewoo-Won@users.noreply.github.com" \
  && git config user.name "Taewoo-Won"
git add . && git commit -m "bootstrap ledger-api with health endpoint (verified: 200 OK)"
git log -1 --format='%ae'                               # noreply 확인
gh repo create ledger-api --public --source=. --push
curl -s "https://api.github.com/repos/Taewoo-Won/ledger-api/commits/main" | grep '"login'"
```

8. 산출물 & 정리 대상

- 신규 repo: <https://github.com/Taewoo-Won/ledger-api> (public)
- 커밋: [94c794a](#) — 13 files / 475 insertions / health endpoint / verified 200 OK
- 핵심 파일: [HealthController.java](#) (PART 2-1 풀코드)
- 정리 대상 (다음 세션, 후반 슬롯): [server.log](#) 가 커밋에 포함됨 → [.gitignore](#) 에 추가해 제외. (mini-ledger 쪽 gitignore 정리 항목과 함께 처리하면 한 번에 끝.)

9. 로드맵 위치

- m105 조각 1 완료 — 첫 Spring Boot API 생성·기동·검증·push·잔디 확인.
- m105 완료 조건 (조각 2, 다음): mini-ledger의 [Bank](#) 를 이 프로젝트에 태워 [POST /transfer](#) 요청 → 잔액 변경 확인. 순수 Java 로직(어제까지의 원장)과 HTTP(오늘의 층)가 만나는 지점.
- IntelliJ clone 시 주의: Gradle sync가 약한 PC엔 무거울 수 있음(Power Save + 인내). 안 되면 편집은 PC·실행은 VPS로 분리.
- 8월 예고: 오늘 목록화한 마법(요청 받는 서버)의 아래층을 Layer 0에서 직접 만든다. 오늘의 마법이 그때 "당연한 구조"로 바뀌는 게 목표.

10. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. [@RestController](#) 와 [@GetMapping\("/health"\)](#) 는 각각 스프링에게 무엇을 시키나? return 값은 뭐가 되나?
2. curl이 무반응처럼 보였던 진짜 원인은? 무엇 두 가지(상태코드·?)로 응답의 실체를 판정했나?
3. [EXECUTING \[16m\]](#) 이 "멈춤이 아닌" 이유는? [java Main](#) 과 서버의 근본 차이는?
4. 16분이 걸린 진짜 원인은? 두 번째 기동은 왜 3.9초였나?
5. [&](#) , [> server.log 2>&1](#) , [kill %1](#) 은 각각 무슨 뜻인가? gradle PID와 java PID가 달랐던 이유는?
6. 검증이 끝나자마자 서버를 내린 이유는?
7. push가 성공해도 잔디가 안 깔릴 수 있다 — 무엇이 일치해야 하고, 무엇([login](#))으로 최종 판정했나?
8. [package com.ledger.api;](#) 는 무엇과 일치해야 하나?
9. 오늘의 커밋 순서 원칙은? (무엇을 확인한 다음에 커밋했나)

11. 회고

오늘 가장 큰 전환은 기능이 아니라 **감각**이다 — "죽지 않는 프로세스"를 처음 다뤘다. [java Main](#) 만 다뤄본 눈에 [EXECUTING \[16m 18s\]](#) 는 영락없는 멈춤이었고, 실제로 "이거 맞냐 16분"이라고 물었다. 아니었다. 서버는 원래 그렇게 산 채로 기다린다. 이 낯설이 풀린 순간이, 오늘 배운 어떤 문법보다 크다. 앞으로 만들 모든 것이 이 "기다리는 프로세스" 위에 올라간다.

오늘은 착시가 세 번 있었고, 세 번 다 같은 방법으로 뚫었다. ① 16분 멈춤처럼 보인 것 → 로그를 읽으니 Gradle 최초 다운로드 + 정상 대기였다. ② curl 무반응처럼 보인 것 → "서버가 덜 뒀다"는 첫 가설을 grep으로 기각하고, [-v](#) 로 해부하니 200 OK와 16바이트가 매번 오고 있었다(개행이 없어 프롬프트에 붙었을 뿐). ③ push 성공 = 잔디라는 가정 → API의 [login](#) 까지 확인해서야 "깎인다"를 확정했다. 어제 원자성 버그를 "가정 말고 실행으로 확인"으로 잡았는데, 오늘 세 착시가 전부 같은 문장으로 풀렸다. 이 문장이 이

를 연속 값을 했다.

순서도 지켰다. 잔디가 급해도 curl 200을 본 다음에 커밋했다. "동작 확인 → 커밋"을 잔디 욕심에 뒤집지 않은 것 — 검증 안 된 반쪽을 올리지 않는 규율이 오늘도 유지됐다.

솔직한 상태 점검: Spring은 아직 마법이다. @ 스티커 하나로 URL이 메서드에 연결되는 내부가 어떻게 되는지는 모른다. 다만 마법을 삼키지 않고 목록으로 만들었다 — Tomcat 기동, 컴포넌트 스캔, URL 매핑. 이 세 개가 "스프링이 나 대신 해준 일"이고, 8월에 이 아래층을 직접 만들 때 오늘의 목록이 지도가 된다. Gradle도 아직 블랙박스다(그냥 bootRun만 쳤다). 지금은 그래도 된다 — 위층을 먼저 세워 내가 뭘 만들어야 하는지 본 것이 오늘의 몫이다.

다음은 조각 2. 어제까지 만든 Bank(원자성 검증, 복식부기까지 들어간 그 원장)를 이 서버에 태워 POST /transfer 로 진짜 이체를 HTTP로 받는다. 순수 Java 여정과 오늘의 HTTP 층이 거기서 처음 만난다. 그게 m105의 완료 조건이다.

한 줄: 순수 Java를 졸업하고 첫 HTTP 서버를 띄운 날 — 16분·curl·잔디, 세 번의 착시를 전부 "가정 말고 확인"으로 뚫고, 검증된 200 OK만 커밋했다.